



UNIVERSIDADE DO ESTADO DO AMAZONAS
ESCOLA SUPERIOR DE TECNOLOGIA

Josué Azevedo Gomes - 2215310054
Matheus Martins Rodrigues - 2215310024
Vitor Hugo Trovão de Moraes - 2215310049

Leitura e Manipulação de Grafos: Labirinto com Busca em Profundidade (DFS)

MANAUS – AMAZONAS
2024

1. Problema

O seguinte trabalho consiste em implementar um programa, em linguagem C, que resolva um labirinto na abordagem de grafos. O labirinto é dado por uma matriz de caracteres ("char") onde 'X' representa uma parede, '0' representa um caminho, 'E' representa a entrada e 'S' representa a saída. O objetivo é imprimir na tela um caminho válido que leve da entrada até a saída, esse caminho não precisa ser o mais curto, mas ele não pode passar duas vezes pela mesma casa.

2. Solução Proposta

A implementação do programa foi realizada em linguagem C, e baseando-se no algoritmo de busca em profundidade (DFS) que percorre um caminho até onde for possível. A solução foi dividida no programa principal, que contém a execução do código, e o programa auxiliar que encapsula a resolução do labirinto.

Sobre a entrada de dados, é feita a leitura de um arquivo .txt, com os caracteres '0' e '1' que representam, respectivamente, os caminhos livres e as paredes do labirinto. O programa manipula os dados na forma de matriz de adjacência, marcando cada posição do labirinto com base no tamanho máximo definido no programa.

Seguindo para a resolução, o programa determina a entrada do labirinto, e assim, percorre os caminhos válidos até encontrar a saída do labirinto. A entrada é determinada durante a execução do código como o primeiro valor '0' encontrado, ou seja, o primeiro caminho livre, e a saída é, por padrão, a última posição do labirinto.

2.1. Funções e Procedimentos do Programa

Para esse programa, foram implementadas as seguintes funções:

Encontrar_caminho: A função percorre um caminho livre na matriz do labirinto. A partir das posições atuais x e y, o processo é executado de forma recursiva até que seja encontrado a saída ou um caminho inválido. Nesta última condição, o código marca o caminho percorrido como 'beco sem saída' e volta para a primeira posição nesse caminho.

C/C++

```
int encontrar_caminho(char labirinto[SIZE][SIZE], int x, int y) {

    // Verificar se a posição atual é a posição final
    if (x == SIZE - 1 && y == SIZE - 1) {
        labirinto[x][y] = 'S'; // Marcar como caminho correto
        return 1;             // Caminho encontrado
    }

    // Verificar se a posição atual é válida
    if (x >= 0 && y >= 0 && x < SIZE && y < SIZE && labirinto[x][y] == '0' ||
        labirinto[x][y] == 'E') {
```

```

// Marcar a posição como caminho percorrido
labirinto[x][y] = '2';

// Tentar mover para a direita
if (encontrar_caminho(labirinto, x, y + 1)) {
    return 1;
}

// Tentar mover para baixo
if (encontrar_caminho(labirinto, x + 1, y)) {
    return 1;
}

// Tentar mover para a esquerda
if (encontrar_caminho(labirinto, x, y - 1)) {
    return 1;
}

// Tentar mover para cima
if (encontrar_caminho(labirinto, x - 1, y)) {
    return 1;
}

// Se nenhum movimento levar ao caminho correto, marcar como beco sem saída
labirinto[x][y] = '4';
return 0;
}

return 0; // Posição inválida
}

```

ler_labirinto: Função que faz a leitura do arquivo .txt do labirinto e armazena na forma de matriz de adjacência. Retorna 1 se o arquivo for aberto e lido com sucesso, caso contrário retorna 0.

```

C/C++

int ler_labirinto(const char *arquivo, char labirinto[SIZE][SIZE]) {
    FILE *file = fopen(arquivo, "r");
    if (file == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 0;
    }
}

```

```

for (int i = 0; i < SIZE; i++) {
    for (int j = 0; j < SIZE; j++) {
        if (fscanf(file, "%c ", &labirinto[i][j]) != 1) {
            printf("Erro ao ler o labirinto do arquivo.\n");
            fclose(file);
            return 0;
        }
    }
}

fclose(file);
return 1;
}

```

imprimir_labirinto: Procedimento que faz a impressão da matriz do labirinto, indicando a entrada ('E'), parede ('X'), caminho ('0') e saída ('S') para cada posição x e y do labirinto. A indicação foi feita usando correlação com valores inteiros, sendo '0' para caminhos livres, '1' para paredes, '2' para caminhos percorridos e '4' para 'beco sem saída'. Além disso, há um tratamento específico definido pelos parâmetros 'entrada_x' e 'entrada_y', que determinam a posição x e y da entrada do labirinto.

C/C++

```

void imprimir_labirinto(char labirinto[SIZE][SIZE], int entrada_x,
                        int entrada_y) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (i == entrada_x && j == entrada_y) {
                printf("E ");
            } else {
                switch (labirinto[i][j]) {
                    case '1':
                        printf("X ");
                        break;
                    case '0':
                        printf("- ");
                        break;
                    case '2':
                        printf("0 ");
                        break;
                    default:
                        printf("%c ", labirinto[i][j]);
                        break;
                }
            }
        }
    }
}

```

```

    }
    printf("\n");
}
}

```

3. Diagrama do Código

